BEE 4750 Homework 4: Linear Programming and Capacity Expansion

Name: Claire Jacobson

ID: cpj35

Due Date

Thursday, 11/06/25, 9:00pm

Overview

Instructions

- Problem 1 asks you to analytically (or graphically) solve a linear program.
- Problem 2 asks you to formulate and solve a resource allocation problem using linear programming.
- Problem 3 asks you to formulate, solve, and analyze a standard generating capacity expansion problem.

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()
```

```
  Activating project at `~/hw4-clairepj`
```

```
using JuMP
using HiGHS
using DataFrames
using Plots
using Measures
using CSV
using MarkdownTables
```

Problems (Total: 30 Points)

Problem 1 (Total: 3 Points)

Solve the following linear program **analytically**. Show your work and justify any reasoning.

$$\begin{aligned} \max_{x,y} \quad & 3x + 2y \\ \text{subject to:} \quad & x + 4y \leq 16 \\ & x - 2y \leq -1 \\ & 0 \leq x \leq 4 \\ & 1 \leq y \leq 4. \end{aligned}$$

Points of intersection: x=4 and y=4 w/ x+4y = 16, x=4 and y=1 w/ x-2y =-1, max can only occur at intersections, evaluate each intersection. Points (4,3), (0,4), (4,2.5), and (1,1), respectively. These points all lie on the corners of the cross-section area which means they are within the constraints of the system. Next I will evaluate the solution to 3x+2y to find the maximum. The values are (3*4)+(2*3) = 18, (3*0)+(2*4)=8, (3*4)+(2*2.5)=17, and (3*1)+(2*1)=5, respectively. Therefore, the maximum of 3x+2y occurs at the point (4,3).

Problem 2 (12 points)

A farmer has access to a pesticide which can be used on corn, soybeans, and wheat fields, and costs $70/kg-yr to apply. The crop yields the farmer can obtain following crop yields by applying varying rates of pesticides to the field are shown in Table 1.

Application Rate (kg/ha)	Soybean (kg/ha)	Wheat (kg/ha)	Corn (kg/ha)
0	2900	3500	5900
1	3800	4100	6700
2	4400	4200	7900

Table 1: Crop yields from applying varying pesticide rates for Problem 1.

The costs of production, *excluding pesticides*, for each crop, and selling prices, are shown in Table 2.

Crop	Production Cost ($/ha-yr)	Selling Price ($/kg)
Soybeans	350	0.36
Wheat	280	0.27
Corn	390	0.22

Table 2: Costs of crop production, excluding pesticides, and selling prices for each crop.

Recently, environmental authorities have declared that farms cannot have an *average* application rate on soybeans, wheat, and corn which exceeds 0.8, 0.7, and 0.6 kg/ha, respectively. The farmer has asked you for advice on how they should plant crops and apply pesticides to maximize profits over 130 total ha while remaining in regulatory compliance if demand for each crop (which is the maximum the market would buy) this year is 250,000 kg?

Problem 2.1

Formulate a linear program for this resource allocation problem, including clear definitions of decision variable(s) (including units), objective function(s), and constraint(s) (make sure to explain functions and constraints with any needed derivations and explanations). **Tip: Make sure that all of your constraints are linear**.

Problem 2.2

How many ha should the farmer dedicate to each crop and with what pesticide application rate(s)? How much profit will the farmer expect to make?

Application Rate (kg/ha)	Soybean (ha)	Wheat (ha)	Corn (ha)
0	13.81	55.25	0.0
1	6.74	15.73	0.0
2	26.92	0.0	11.53

Profit: $116,741.17

Problem 2.3

The farmer has an opportunity to buy an extra 10 ha of land. How much extra profit would this land be worth to the farmer? Discuss why this value makes sense and under what conditions you would recommend the farmer should make the purchase.

The new profit would be $124,035.17 meaning the addition of 10 acres of land would lead to a little over $7,000 increase in profit. This addition would only be logical if this benefit outweighs the cost of the purchase of the land itself. Further, the farmer should consider if they have the capacity to work on 10 more ha of land (though in theory this is already accounted for in the cost of cultivating the crops).

```
#2.1

#table of all values for each crop

data = DataFrame(
    crop = ["Soy", "Wheat", "Corn"],
    yield_0 = [2900, 3500, 5900],
    yield_1 = [3800, 4100, 6700],
    yield_2 = [4400, 4200, 7900],
    cost_prod = [350, 280, 390],
    profit = [0.36, 0.27, 0.22],
    max_app = [0.8, 0.7, 0.6]
)

#constants/limits
pesticide_cost =70
ha_max =140
dmd = 250_000
rates = 0:2

rows = nrow(data)

crop_model = Model(HiGHS.Optimizer) # initialize model object
@variable(crop_model, ha[1:rows,0:2] >= 0) # non-negativity constraints

@expression(crop_model, revenue,
    sum(data.profit[i] * data[!, Symbol("yield_$(r)")][i] * ha[i, r] for i in 1:rows, r in ra
)
@expression(crop_model, prod_cost,
    sum(data.cost_prod[i] * ha[i, r] for i in 1:rows, r in rates)
)
@expression(crop_model, pesticide_spent,
```

```
    sum(pesticide_cost * r * ha[i, r] for i in 1:rows, r in rates)
)

@objective(crop_model, Max, revenue - prod_cost - pesticide_spent)

@constraint(crop_model, sum(ha[i, r] for i in 1:rows, r in rates) <= ha_max)
@constraint(crop_model, [i in 1:rows],sum(data[!, Symbol("yield_$(r)")][i] * ha[i, r] for r i
@constraint(crop_model, [i in 1:rows], sum(r * ha[i, r] for r in rates) <= data.max_app[i] *

optimize!(crop_model)

print(crop_model) # this outputs a nicely formatted summary of the model so you can check you
```

```
Running HiGHS 1.11.0 (git hash: 364c83a51e): Copyright (c) 2025 HiGHS under MIT licence terms
LP   has 7 rows; 9 cols; 27 nonzeros
Coefficient ranges:
  Matrix [2e-01, 8e+03]
  Cost   [7e+02, 1e+03]
  Bound  [0e+00, 0e+00]
  RHS    [1e+02, 2e+05]
Presolving model
7 rows, 9 cols, 27 nonzeros  0s
7 rows, 9 cols, 27 nonzeros  0s
Presolve : Reductions: rows 7(-0); columns 9(-0); elements 27(-0) - Not reduced
Problem not reduced by presolve: solving the LP
Using EKK dual simplex solver - serial
  Iteration        Objective     Infeasibilities num(sum)
          0    -1.8170298458e+02 Ph1: 7(28.3601); Du: 9(181.703) 0s
          7     1.2403516702e+05 Pr: 0(0) 0s
Model status        : Optimal
Simplex   iterations: 7
Objective value     :  1.2403516702e+05
P-D objective error :  1.1732040935e-16
HiGHS run time      :          0.03
```

$$
\begin{aligned}
\max \quad & 694ha_{1,0} + 948ha_{1,1} + 1094ha_{1,2} + 665.0000000000001ha_{2,0} + 757ha_{2,1} + 714ha_{2,2} + 908ha_{3,0} + 101 \\
\text{Subject to} \quad & ha_{1,0} + ha_{2,0} + ha_{3,0} + ha_{1,1} + ha_{2,1} + ha_{3,1} + ha_{1,2} + ha_{2,2} + ha_{3,2} \leq 140 \\
& 2900ha_{1,0} + 3800ha_{1,1} + 4400ha_{1,2} \leq 250000 \\
& 3500ha_{2,0} + 4100ha_{2,1} + 4200ha_{2,2} \leq 250000 \\
& 5900ha_{3,0} + 6700ha_{3,1} + 7900ha_{3,2} \leq 250000 \\
& -0.8ha_{1,0} + 0.19999999999999996ha_{1,1} + 1.2ha_{1,2} \leq 0 \\
& -0.7ha_{2,0} + 0.30000000000000004ha_{2,1} + 1.3ha_{2,2} \leq 0 \\
& -0.6ha_{3,0} + 0.4ha_{3,1} + 1.4ha_{3,2} \leq 0 \\
& ha_{1,0} \geq 0 \\
& ha_{2,0} \geq 0 \\
& ha_{3,0} \geq 0 \\
& ha_{1,1} \geq 0 \\
& ha_{2,1} \geq 0 \\
& ha_{3,1} \geq 0 \\
& ha_{1,2} \geq 0 \\
& ha_{2,2} \geq 0 \\
& ha_{3,2} \geq 0
\end{aligned}
$$

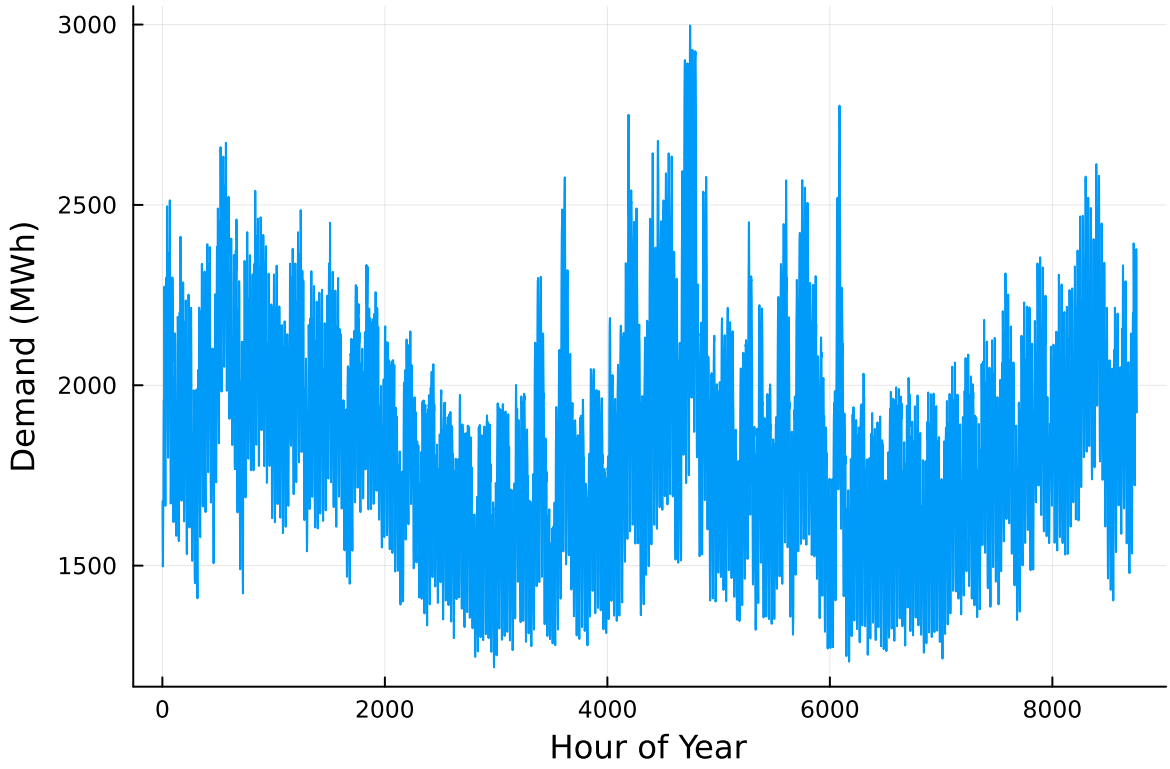
```
#2.2
@show("crops and application values")
@show value.(ha);

@show("net profit from crops:")
@show objective_value(crop_model);
```

```
"crops and application values" = "crops and application values"
value.(ha) = 2-dimensional DenseAxisArray{Float64,2,...} with index sets:
    Dimension 1, Base.OneTo(3)
    Dimension 2, 0:2
And data, a 3×3 Matrix{Float64}:
 13.81215469613259   55.2486187845304     0.0
  9.743306417339566  22.734381640458984   0.0
 26.923076923076923   0.0                 11.538461538461522
"net profit from crops:" = "net profit from crops:"
objective_value(crop_model) = 124035.16702082446
```

Problem 3 (15 points)

For this problem, we will use hourly load (demand) data from 2013 in New York's Zone C (which includes Ithaca). The load data is loaded and plotted below in Figure 1.

```
# load the data, pull Zone C, and reformat the DataFrame
NY_demand = DataFrame(CSV.File("data/2013_hourly_load_NY.csv"))
rename!(NY_demand, :"Time Stamp" => :Date)
demand = NY_demand[:, [:Date, :C]]
rename!(demand, :C => :Demand)
demand[:, :Hour] = 1:nrow(demand)

# plot demand
plot(demand.Hour, demand.Demand, xlabel="Hour of Year", ylabel="Demand (MWh)", label=:false)
```



Next, we load the generator data, shown in Table 3. This data includes fixed costs ($/MW installed), variable costs ($/MWh generated), and CO2 emissions intensity (tCO2/MWh generated).

```
gens = DataFrame(CSV.File("data/generators.csv"))
```

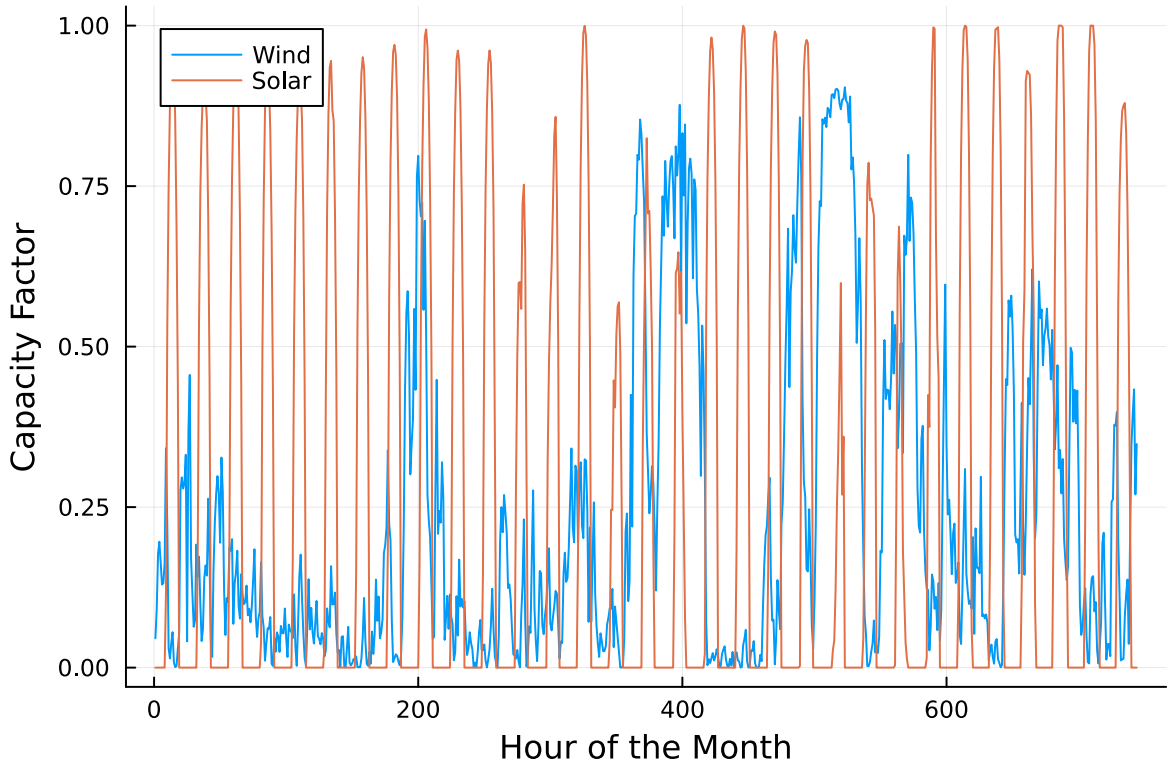
	Plant	FixedCost	VarCost	Emissions
	String15	Int64	Int64	Float64
1	Geothermal	450000	0	0.0
2	Coal	220000	24	1.0
3	NG CCGT	82000	30	0.43
4	NG CT	65000	40	0.55
5	Wind	91000	0	0.0
6	Solar	70000	0	0.0

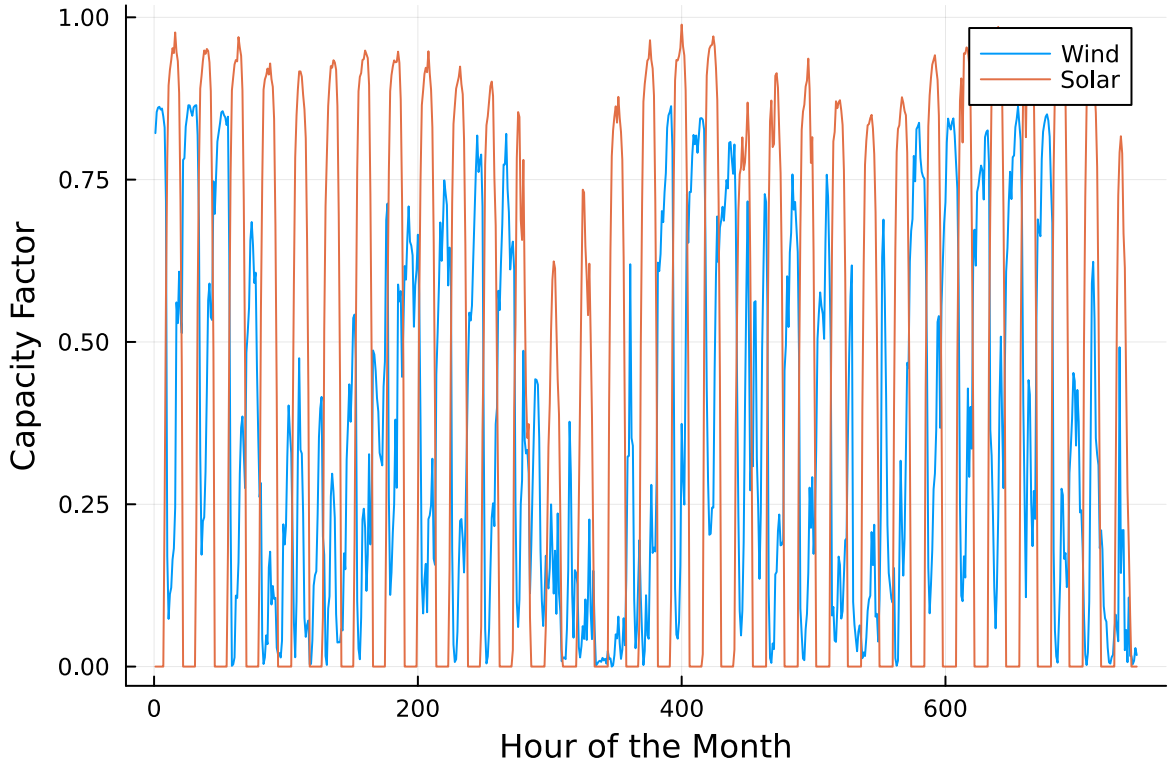
Finally, we load the hourly solar and wind capacity factors, which are plotted in Figure 2. These tell us the fraction of installed capacity which is expected to be available in a given hour for generation (typically based on the average meteorology).

```
# load capacify factors into a DataFrame
cap_factor = DataFrame(CSV.File("data/wind_solar_capacity_factors.csv"))

# plot January capacity factors
p1 = plot(cap_factor.Wind[1:(24*31)], label="Wind")
plot!(cap_factor.Solar[1:(24*31)], label="Solar")
xaxis!("Hour of the Month")
yaxis!("Capacity Factor")

p2 = plot(cap_factor.Wind[4344:4344+(24*31)], label="Wind")
plot!(cap_factor.Solar[4344:4344+(24*31)], label="Solar")
xaxis!("Hour of the Month")
yaxis!("Capacity Factor")

display(p1)
display(p2)
```

You have been asked to develop a generating capacity expansion plan for the utility in Riley County, NY, which currently has no existing electrical generation infrastructure. The utility can build any of the following plant types: geothermal, coal, natural gas combined cycle gas turbine (CCGT), natural gas combustion turbine (CT), solar, and wind. The NY state legislature is planning to enact an annual CO2 limit, which for the utility would limit the emissions in its footprint to 1.5 MtCO2/yr.

While coal, CCGT, and CT plants can generate at their full installed capacity, geothermal plants operate at maximum 85% capacity, and solar and wind available capacities vary by the hour depend on the expected meteorology. The utility will also penalize any non-served demand at a rate of $10,000/MWh.


Wind
Solar
1.00
0.75
0.50
0.25
0.00
Capacity Factor
0
200
400
600
Hour of the Month


Wind
Solar
1.00
0.75
0.50
0.25
0.00
Capacity Factor
0
200
400
600
Hour of the Month

Problem 3.1

Formulate a linear program for this capacity expansion problem, including clear definitions of decision variable(s) (including units), objective function(s), and constraint(s) (make sure to explain functions and constraints with any needed derivations and explanations).

Problem 3.2

How much should the utility build of each type of generating plant? What will the total cost be? How much energy will be non-served?

Problem 3.3

What fraction of annual generation does each plant type produce? How does this compare to the breakdown of built capacity that you found in Problem 3.2? Do these results make sense given the demand and generator data?

Problem 3.4

Make a plot of the electricity price in each hour. Discuss any trends that you see.

Problem 3.5

What is the shadow price of CO2? What is the value to the utility be of allowing it to emit an additional 1000 tCO2/yr?

Significant Digits

Use `round(x; digits=n)` to report values to the appropriate precision! If your number is on a different order of magnitude and you want to round to a certain number of significant digits, you can use `round(x; sigdigits=n)`.

Getting Variable Output Values

`value.(x)` will report the values of a `JuMP` variable `x`, but it will return a special container which holds other information about `x` that is useful for `JuMP`. This means that you can't use this output directly for further calculations. To just extract the values, use `value.(x).data`.

Suppressing Model Command Output

The output of specifying model components (variable or constraints) can be quite large for this problem because of the number of time periods. If you end a cell with an `@variable` or `@constraint` command, I *highly* recommend suppressing

output by adding a semi-colon after the last command, or you might find that your notebook crashes.

```
names(gens)
first(gens, 5)
```

	Plant	FixedCost	VarCost	Emissions
	String15	Int64	Int64	Float64
1	Geothermal	450000	0	0.0
2	Coal	220000	24	1.0
3	NG CCGT	82000	30	0.43
4	NG CT	65000	40	0.55
5	Wind	91000	0	0.0

```
names(gens)
```

```
4-element Vector{String}:
 "Plant"
 "FixedCost"
 "VarCost"
 "Emissions"
```

```
#3.1

techs = Vector(gens[!, :Plant])
hours = 1:8760

FixedCost = Dict(zip(techs, gens[!,:FixedCost]))
VarCost = Dict(zip(techs, gens[!,:VarCost]))
E = Dict(zip(techs, gens[!,:Emissions]))

capacity_model=Model(HiGHS.Optimizer)

@variable(capacity_model, C[i in techs] >= 0)          # capacity [MW]
@variable(capacity_model, g[i in techs, t in hours] >= 0)  # generation [MWh]
@variable(capacity_model, u[t in hours] >= 0)          # unmet demand [MWh]

@constraint(capacity_model, [t in hours], sum(g[i,t] for i in techs) + u[t] == demand[t])
@constraint(capacity_model, [i in techs, t in hours], g[i,t] <= C[i] * avail[i,t])
@constraint(capacity_model, sum(E[i] * g[i,t] for i in techs, t in hours) <= 1.5e6)

@objective(capacity_model, Min, sum(FixedCost[i]*C[i] for i in techs)
```

```
                    + sum(VarCost[i]*g[i,t] for i in techs, t in hours)
                    + sum(10000*u[t] for t in hours))
```

```
LoadError: ArgumentError: syntax df[column] is not supported use df[!, column] instead
ArgumentError: syntax df[column] is not supported use df[!, column] instead
Stacktrace:
  [1] getindex(::DataFrame, ::Int64)
    @ DataFrames ~/.julia/packages/DataFrames/b4w9K/src/abstractdataframe/abstractdataframe.j
  [2] macro expansion
    @ ~/.julia/packages/MutableArithmetics/VMS0x/src/rewrite.jl:374 [inlined]
  [3] macro expansion
    @ ~/.julia/packages/JuMP/vfM5V/src/macros.jl:264 [inlined]
  [4] macro expansion
    @ ~/.julia/packages/JuMP/vfM5V/src/macros/@constraint.jl:171 [inlined]
  [5] (::var"#103#104"{Model})(t::Int64)
    @ Main ~/.julia/packages/JuMP/vfM5V/src/Containers/macro.jl:559
  [6] #87
    @ ~/.julia/packages/JuMP/vfM5V/src/Containers/container.jl:124 [inlined]
  [7] iterate
    @ ./generator.jl:48 [inlined]
  [8] collect(itr::Base.Generator{JuMP.Containers.VectorizedProductIterator{Tuple{UnitRange{
    @ Base ./array.jl:791
  [9] map(f::Function, A::JuMP.Containers.VectorizedProductIterator{Tuple{UnitRange{Int64}}}
    @ Base ./abstractarray.jl:3399
 [10] container(f::Function, indices::JuMP.Containers.VectorizedProductIterator{Tuple{UnitRa
    @ JuMP.Containers ~/.julia/packages/JuMP/vfM5V/src/Containers/container.jl:123
 [11] container(f::Function, indices::JuMP.Containers.VectorizedProductIterator{Tuple{UnitRa
    @ JuMP.Containers ~/.julia/packages/JuMP/vfM5V/src/Containers/container.jl:75
 [12] macro expansion
    @ ~/.julia/packages/JuMP/vfM5V/src/macros.jl:402 [inlined]
 [13] top-level scope
    @ In[42]:16
```

References

List any external references consulted, including classmates.